

中南大学2024年暑期数学建模竞赛培训

偏微分方程的计算方法简介

中南大学 数学与统计学院

2024年08月

目录

- 1 快速计算方法
 - 差分逼近导数
 - 迎风格式
 - 加快计算过程
- 2 预估-校正格式
 - 一般构造方式
 - 数值实验

数值求解偏微分方程的一般步骤

- 确定模型的守恒律方程、定解条件
- 选择合适的求解区域、并离散化方程和定解条件
- 写出对应的离散方程
- 求解线性或非线性方程组(直接法、迭代法)

抛物型方程有限差分格式

考虑扩散方程

$$\frac{\partial u(x, t)}{\partial t} = \nu^2 \frac{\partial^2 u(x, t)}{\partial x^2} + f(x, t), \quad (x, t) \in [0, L] \times [0, T], \quad (1)$$

定解条件取为

$$\begin{aligned} u(x, 0) &= u_0(x), \quad 0 \leq x \leq L, \\ u(0, t) &= u(L, t) = 0, \quad t > 0. \end{aligned}$$

抛物型方程有限差分格式

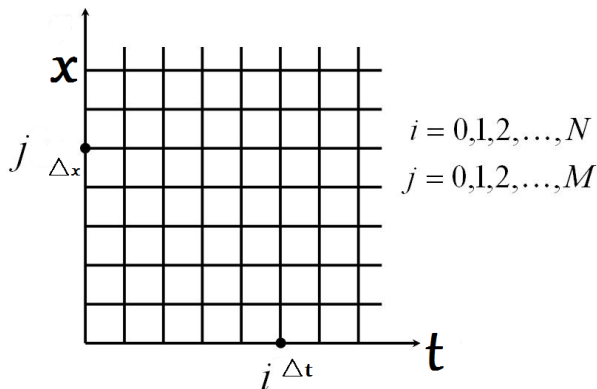


图1. 离散网格示意图¹

¹<https://cn.bing.com/images/>

抛物型方程有限差格式

为方便讨论, 我们引入如下记号, $u_j^i = u(t_i, x_j)$:

用 u_j^i 表示函数 $u(t, x)$ 在 $t = t_i$ 和 $x = x_j$ 处的函数值.

则有

$$\frac{u_j^{i+1} - u_j^i}{\Delta t} = \nu^2 \frac{u_{j+1}^{i+1} - 2u_j^{i+1} + u_{j-1}^{i+1}}{(\Delta x)^2} + f(x_j, t_{i+1}), \quad (2)$$

以及

$$\frac{u_j^{i+1} - u_j^i}{\Delta t} = \nu^2 \frac{u_{j+1}^i - 2u_j^i + u_{j-1}^i}{(\Delta x)^2} + f(x_j, t_i), \quad (3)$$

抛物型方程有限差格式

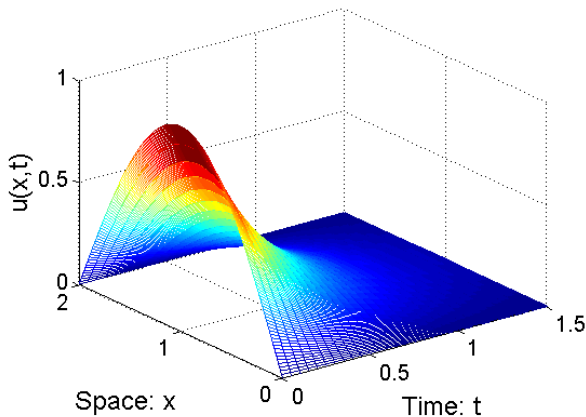


图2. 数值解示意图²

椭圆型方程有限差分格式

考虑Poisson方程

$$-\Delta u = f(x, y), \quad (x, y) \in [0, 1] \times [0, 1], \quad (4)$$

其中

$$f(x, y) = \sin(\pi x) \sin(\pi y) + \sin(\pi x) \sin(2\pi y).$$

定解条件取为

$$\begin{aligned} u(x, 0) &= u(x, 1) = 0, \quad 0 \leq x \leq 1, \\ u(0, y) &= u(1, y) = 0, \quad 0 \leq y \leq 1. \end{aligned}$$

椭圆型方程有限差格式

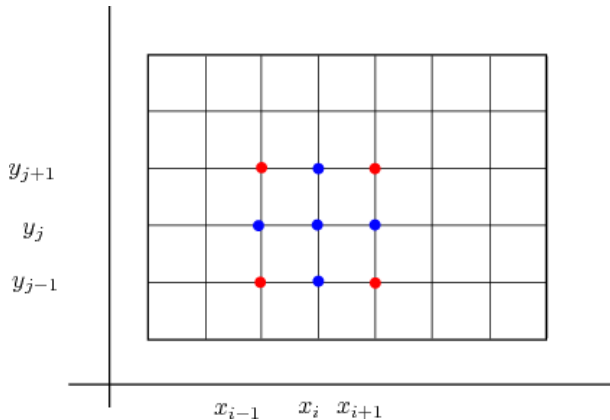


图3. 离散网格示意图³

³<https://cn.bing.com/images/>

椭圆型方程有限差格式

类似地, 引入如下记号, $u_j^i = u(x_i, y_j)$:

用 u_j^i 表示函数 $u(x, y)$ 在 $x = x_i$ 和 $y = y_j$ 处的函数值.

则有

$$-\left[\frac{u_j^{i+1} - 2u_j^i + u_j^{i-1}}{h^2} + \frac{u_{j+1}^i - 2u_j^i + u_{j-1}^i}{h^2} \right] = f(x_i, y_j), \quad (5)$$

即

$$u_j^i = \frac{1}{4} \left(u_j^{i+1} + u_j^{i-1} + u_{j+1}^i + u_{j-1}^i \right) + \frac{h^2}{4} f(x_i, y_j), \quad (6)$$

这意味着 u_j^i 总是用相邻的四个点的平均值来实现的.

椭圆型方程有限差格式

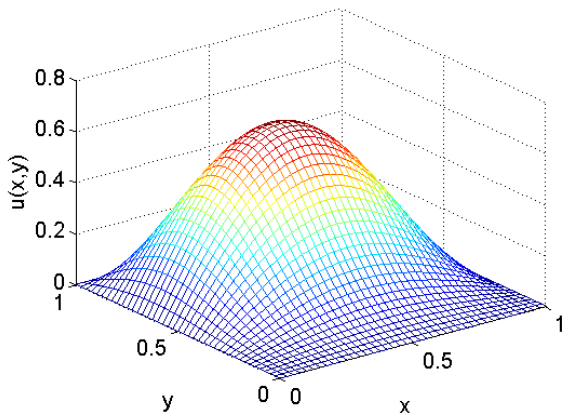


图4. 数值解示意图⁴

⁴Generated by MATLAB 7.0

迎风格式

现在我们回到追赶问题

$$\frac{\partial u(t, x)}{\partial t} + a \frac{\partial u(t, x)}{\partial x} = 0,$$

这个方程虽然简单, 但对于研究差分格式的构造是很有启发的.

双曲方程与椭圆方程和抛物方程的重要区别, 是双曲方程具有特征和特征关系. 初值的函数性质(如间断, 弱间断等)将沿着特征传播, 因而解一般无光滑性.

为方便讨论, 仍用如下记号, $u_j^n = u(t_n, x_j)$:

用 u_j^n 表示函数 $u(t, x)$ 在 $t = t_n$ 和 $x = x_j$ 处的函数值.

迎风格式

利用差商近似导数, 我们可得方程(1)的如下三种格式:

$$\frac{u_j^{n+1} - u_j^n}{\tau} + a \frac{u_j^n - u_{j-1}^n}{h} = 0, \quad (7)$$

$$\frac{u_j^{n+1} - u_j^n}{\tau} + a \frac{u_{j+1}^n - u_j^n}{h} = 0, \quad (8)$$

$$\frac{u_j^{n+1} - u_j^n}{\tau} + a \frac{u_{j+1}^n - u_{j-1}^n}{2h} = 0. \quad (9)$$

前两个方程的截断误差阶为 $\mathcal{O}(\tau + h)$, 第三个方程的截断误差阶为 $\mathcal{O}(\tau + h^2)$.

下面我们探讨这三种格式的适用条件. 首先令

$r = \frac{\tau}{h}$, 表示时间步长和空间步长的比值.

迎风格式

数值格式(7), (8)和(9)可以表示为:

$$u_j^{n+1} = ar u_{j-1}^n + (1 - ar) u_j^n, \quad (10)$$

$$u_j^{n+1} = (1 + ar) u_j^n - ar u_{j+1}^n, \quad (11)$$

$$u_j^{n+1} = u_j^n + \frac{ar}{2} u_{j-1}^n - \frac{ar}{2} u_{j+1}^n. \quad (12)$$

按照**Fourier分析法**, 将

$$u_j^n = v^n e^{i\alpha x_j}, \quad i = \sqrt{-1}, \quad x_j = jh, \quad \alpha \text{ 是任意实数}$$

代入(10), (11)和(12), 则分别得到

$$v^{n+1} = \left[ar e^{-i\alpha h} + (1 - ar) \right] v^n = \lambda_1 v^n, \quad (13)$$

$$v^{n+1} = \left[(1 + ar) - ar e^{i\alpha h} \right] v^n = \lambda_2 v^n, \quad (14)$$

$$v^{n+1} = [1 - iar \sin(\alpha h)] v^n = \lambda_3 v^n. \quad (15)$$

迎风格式

注意到

$$|\lambda_3| = \sqrt{1 + a^2 r^2 \sin^2(\alpha h)}$$

对任何 $r \neq 0$ 都不满足 von Neumann 条件, 故(9)总是**不稳定**的. 另外, $|\lambda_1| \leq 1$ 等价于 $a^2 r^2 \leq ar$, 即就是 $(a\tau/h)^2 \leq (a\tau/h)$, 从而

- 格式(7)是**稳定**的充要条件为

$$a \geq 0, \quad |ar| \leq 1.$$

- 格式(8)是**稳定**的充要条件为

$$a \leq 0, \quad |ar| \leq 1.$$

这说明: $a \geq 0$ 时只有格式(7)可用, 当 $a \leq 0$ 时只有格式(8)可用.

数值算例

算例1: (双曲型方程)

考虑一阶线性方程

$$\frac{\partial u(t, x)}{\partial t} + a \frac{\partial u(t, x)}{\partial x} = 0, \quad (16)$$

的几种特殊情形: 初值取为 $u(x, 0) = e^{-100(x-1)^2}$, 系数分别为

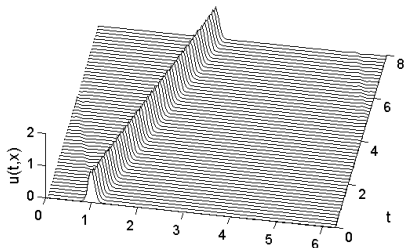
- $a = 0.2$;
- $a = 0.5$;
- $a = 0.2 + \sin^2(x - 1)$;
- $a = 0.01 + \sin^2(x - 1)$.

计算时, 边界条件取为零, 称为人工边界条件.

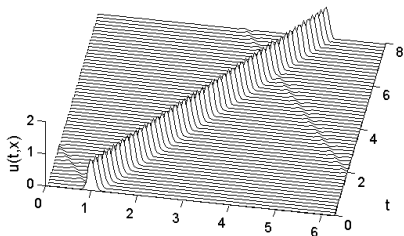
数值算例

$a = 0.2$ 和 $a = 0.5$:

$a = 0.2$



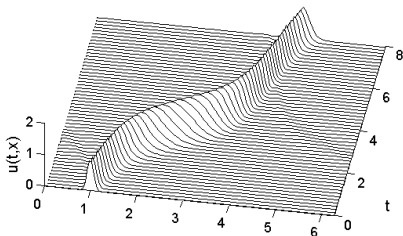
$a = 0.5$



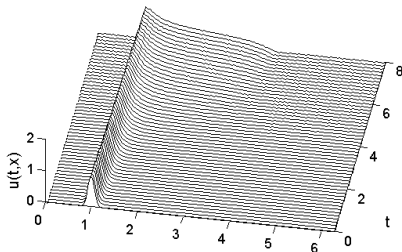
数值算例

$a = 0.2 + \sin^2(x-1)$ 和 $a = 0.01 + \sin^2(x-1)$:

$$a = 0.2 + \sin^2(x-1)$$



$$a = 0.01 + \sin^2(x-1)$$



数值算例

如果遇到了人群相对移动的情形, 则模型转化为:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + \frac{\partial^3 u}{\partial x^3} = 0, \quad (17)$$

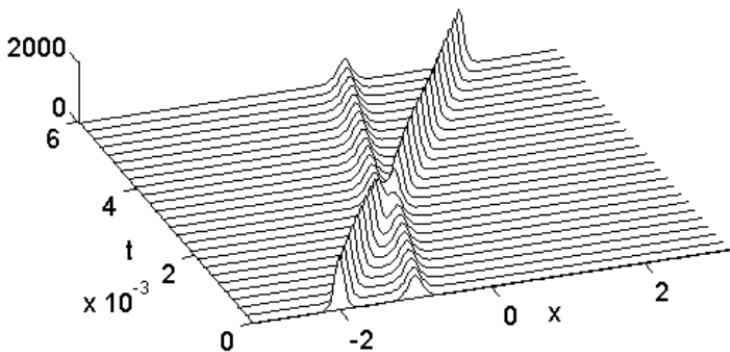
其中 $-\pi \leq x \leq \pi$. 初始值取为

$$u(x, 0) = 3a^2 \operatorname{sech}^2 \left(\frac{a}{2}(x+2) \right) + 3b^2 \operatorname{sech}^2 \left(\frac{b}{2}(x+1) \right).$$

称为KdV方程. 计算时, 边界条件取为零, 称为人工边界条件.

数值算例

取 $a = 25$ 和 $b = 16$, 人群相向而行, 可得:



若干技巧

- 快速迭代
- 矩阵计算代替逐点迭代

$$\frac{dy}{dx} = f(x, y) \quad \Rightarrow \quad Ax = b$$

- 快速Fourier变换(FFT)
- 快速多极算法
- 其他近似算法

迭代法

- 当网格剖分很密时, 方程(2)(3)(6)(7)(8)(9)系数矩阵条件数增加, 直接求解将十分耗时, 且失去精度.
- 解决方案
 - 雅可比迭代法(Jacobi iterative method)
 - 高斯-赛德尔迭代法(Gauss-Seidel iterative method)
 - 超松弛迭代法(SOR iterative method)
 - 共轭梯度法(Conjugate gradient method, 本质上是一种直接法)

Every problem has a solution.

通常来说, 在计算机上设计一种数值算法, 目标都是希望将复杂转化为简单可重复; 或者, 通过简单的重复(迭代, 递归等)逼近原始问题. 计算机擅长简单重复的工作.

直接解法

- 有限次运算获得精确解
- Gauss消元, LU分解等各种矩阵分解技术
- 系数矩阵低维稠密
- 系数矩阵非奇异

迭代解法

- 逐渐收敛的逼近过程, 一般不能有限次求得精确解
- 雅可比迭代法, 高斯-赛德尔迭代法, 超松弛迭代法, 共轭梯度法, Krylov子空间方法
- 系数矩阵高维稀疏
- 系数矩阵具有特殊结构

预备知识

什么是迭代法?

考虑线性方程组

$$\begin{cases} 8x_1 - 3x_2 + 2x_3 = 20, \\ 4x_1 + 11x_2 - x_3 = 33, \\ 6x_1 + 3x_2 + 12x_3 = 36. \end{cases}$$

简记为 $Ax = b$, 其中

$$A = \begin{bmatrix} 8 & -3 & 2 \\ 4 & 11 & -1 \\ 6 & 3 & 12 \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}, \quad b = \begin{bmatrix} 20 \\ 33 \\ 36 \end{bmatrix}$$

不难求得, 方程组的解为

$$x^* = [3, 2, 1]^T.$$

预备知识

什么是迭代法?

可以将线性方程组改写为

$$\begin{cases} x_1 = \frac{1}{8}(3x_2 - 2x_3 + 20), \\ x_2 = \frac{1}{11}(-4x_1 + x_3 + 33), \\ x_3 = \frac{1}{12}(-6x_1 - 3x_2 + 36). \end{cases}$$

简记为 $x = Bx + f$, 其中

$$B = \begin{bmatrix} 0 & \frac{3}{8} & -\frac{2}{8} \\ -\frac{4}{11} & 0 & \frac{1}{11} \\ -\frac{6}{12} & -\frac{3}{12} & 0 \end{bmatrix}, \quad f = \begin{bmatrix} \frac{20}{8} \\ \frac{33}{11} \\ \frac{36}{12} \end{bmatrix}.$$

- 选择 $x^{(0)}$, 比如 $x^{(0)} = [0, 0, 0]^T$, 计算 $x^{(1)} = Bx^{(0)} + f$;
- 将 $x^{(1)}$ 代入右边, 计算 $x^{(2)} = Bx^{(1)} + f$;
- 一直进行下去, 计算 $x^{(n+1)} = Bx^{(n)} + f$.

预备知识

数值实验

方程的准确解为

$$x^* = [3, 2, 1]^T.$$

具体迭代过程如下

n	$x_1^{(0)}$	$x_2^{(0)}$	$x_3^{(0)}$	误差, 2-范数
0	0	0	0	0
1	2.5000000	3.0000000	3.0000000	2.291287847477920
2	2.8750000	2.3636364	1.0000000	0.384521007174741
3	3.1363636	2.0454545	0.9715909	0.146520419979859
4	3.0241477	1.9478306	0.9204545	0.098143978977990
5	3.0003228	1.9839876	1.0009685	0.016044907095934
6	2.9937532	1.9999707	1.0038417	0.007333590729335
7	2.9990286	2.0026208	1.0031307	0.004196864797871
8	3.0002001	2.0006379	0.9998305	0.000689662704929
9	3.0002816	1.9999118	0.9997405	0.000392948226221

预备知识

迭代法的收敛性分析

为简单起见, 将线性方程组记为

$$Ax = b,$$

其中 A 是 n 阶非奇异矩阵, b 为 n 维非零列向量. 为了建立迭代法, 将 A 分裂为

$$A = M - N,$$

其中 M 为可选择的非奇异矩阵, 且使得 $Mx = d$ 容易求解. 称这样的 M 为分裂矩阵.

这样, 求解 $Ax = b$ 就变成了 $d = Mx = Nx + b$, 即就是

$$Ax = b \Leftrightarrow x = M^{-1}Nx + M^{-1}b.$$

预备知识

迭代法的收敛性分析

于是, 要解线性方程组

$$x = Bx + f,$$

可以通过构造如下迭代(称为一阶定常迭代)法:

$$\begin{cases} x^{(0)}, & \text{初始向量, 有时就取为零向量.} \\ x^{(k+1)} = Bx^{(k)} + f, & k = 0, 1, 2, \dots, \end{cases} \quad (18)$$

其中

$$B = M^{-1}N = M^{-1}(M - A) = I - M^{-1}A, \quad f = M^{-1}b.$$

称 $B = I - M^{-1}A$ 为迭代法的迭代矩阵. 选取不同的 M , 就得到解线性方程组 $Ax = b$ 的不同类型的迭代法.

雅可比迭代法

不使用变量的最新信息

雅可比迭代法

$$\left\{ \begin{array}{l} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1, \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2n}x_n = b_2, \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + \cdots + a_{3n}x_n = b_3, \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \cdots + a_{nn}x_n = b_n, \end{array} \right.$$

- 找到对角元;
- 其余移项右边;
- 构造迭代格式.

雅可比迭代法

对应的算法为

$$\begin{cases} x^{(0)} = [x_1^{(0)}, x_2^{(0)}, \dots, x_n^{(0)}]^\top, \\ x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1, j \neq i}^n a_{ij} x_j^{(k)} \right), \end{cases}$$

其中 $i = 1, 2, \dots, n$, $k = 0, 1, 2, \dots$ 表示迭代次数.

雅可比迭代法

求解方程组

$$\begin{cases} x_1 + 2x_2 - 2x_3 = 5, \\ x_1 + x_2 + x_3 = 1, \\ 2x_1 + 2x_2 + x_3 = 3. \end{cases}$$

不难看到, 系数矩阵可分解为

$$\begin{aligned} A &= \begin{bmatrix} 1 & 2 & -2 \\ 1 & 1 & 1 \\ 2 & 2 & 1 \end{bmatrix} \\ &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 \\ -1 & 0 & 0 \\ -2 & -2 & 0 \end{bmatrix} - \begin{bmatrix} 0 & -2 & 2 \\ 0 & 0 & -1 \\ 0 & 0 & 0 \end{bmatrix} \\ &\equiv D - L - U. \end{aligned}$$

雅可比迭代法

Matlab code

```
for i = 1:n
    for j = 1:n
        if i==j
            D(i,i) = A(i,i);
        else if i<j
            U(i,j) = -A(i,j);
        else if i>j
            L(i,j) = -A(i,j);
        end
    end
end
end

for k = 1:N S(:,k+1) = (L+U)*S(:,k) + [5 1 3]'; end
```

雅可比迭代法

于是, 迭代矩阵实际上就是

$$\mathcal{J} = \textcolor{blue}{L} + \textcolor{red}{U} = \begin{bmatrix} 0 & \textcolor{red}{-2} & \textcolor{red}{2} \\ \textcolor{blue}{-1} & 0 & \textcolor{red}{-1} \\ \textcolor{blue}{-2} & \textcolor{blue}{-2} & 0 \end{bmatrix}.$$

直接计算可以验证 $\rho(\mathcal{J}) = \max\{0, 0, 0\} < 1$ (特征值0是三重根), 故雅可比迭代法收敛. 取初始值为 $x^{(0)} = [0, 0, 0]^\top$, 可得

迭代	0	1	2	3	4	5	...
分量 x_1	0	5	9	$\textcolor{red}{1}$	$\textcolor{red}{1}$	$\textcolor{red}{1}$	...
分量 x_2	0	1	-7	$\textcolor{red}{1}$	$\textcolor{red}{1}$	$\textcolor{red}{1}$	...
分量 x_3	0	3	-9	$\textcolor{red}{-1}$	$\textcolor{red}{-1}$	$\textcolor{red}{-1}$	...

第3次迭代就已经得到原方程的准确解(准确解为 $[1, 1, -1]^\top$).

高斯-赛德尔迭代法

使用变量的最新信息

高斯-赛德尔迭代法

$$\left\{ \begin{array}{l} a_{11}x_1 + a_{12}x_2 + a_{13}x_3 + \cdots + a_{1n}x_n = b_1, \\ a_{21}x_1 + a_{22}x_2 + a_{23}x_3 + \cdots + a_{2n}x_n = b_2, \\ a_{31}x_1 + a_{32}x_2 + a_{33}x_3 + \cdots + a_{3n}x_n = b_3, \\ \vdots \\ a_{n1}x_1 + a_{n2}x_2 + a_{n3}x_3 + \cdots + a_{nn}x_n = b_n, \end{array} \right.$$

- 找到对角元;
- 其余移项右边;
- 构造迭代格式(计算 $x_j^{(k)}$ 时, 用上了全部 $x_i^{(k)}$ ($1 \leq i \leq j-1$)).
- **区别:** 雅可比迭代法是每一层独立计算 $x^{(k-1)} \Rightarrow x^{(k)}$.

高斯-赛德尔迭代法

其中蓝色部分在迭代上层方程时未知数已更新, 对应的算法为

$$\begin{cases} x^{(0)} = [x_1^{(0)}, x_2^{(0)}, \dots, x_n^{(0)}]^\top, \\ x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right), \end{cases}$$

其中 $i = 1, 2, \dots, n$, $k = 0, 1, 2, \dots$ 表示迭代次数.

高斯-赛德尔迭代法

求解方程组

$$\begin{cases} 5x_1 + 2x_2 + x_3 = -12, \\ -x_1 + 4x_2 + 2x_3 = 20, \\ 2x_1 - 3x_2 + 10x_3 = 3. \end{cases}$$

不难看到, 系数矩阵可分解为

$$\begin{aligned} A &= \begin{bmatrix} 5 & 2 & 1 \\ -1 & 4 & 2 \\ 2 & -3 & 10 \end{bmatrix} \\ &= \begin{bmatrix} 5 & 0 & 0 \\ 0 & 4 & 0 \\ 0 & 0 & 10 \end{bmatrix} - \begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 0 \\ -2 & 3 & 0 \end{bmatrix} - \begin{bmatrix} 0 & -2 & -1 \\ 0 & 0 & -2 \\ 0 & 0 & 0 \end{bmatrix} \\ &\equiv D - L - U. \end{aligned}$$

高斯-赛德尔迭代法

于是, 迭代矩阵实际上就是

$$\mathcal{G} = (D - \textcolor{blue}{L})^{-1}\textcolor{red}{U} = \begin{bmatrix} 0 & -0.4000 & -0.2000 \\ 0 & -0.1000 & -0.5500 \\ 0 & 0.0500 & -0.1250 \end{bmatrix}.$$

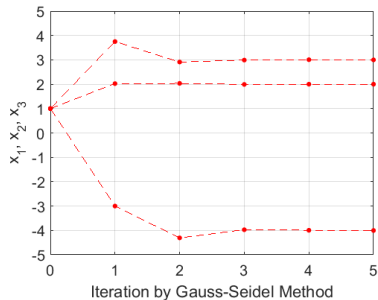
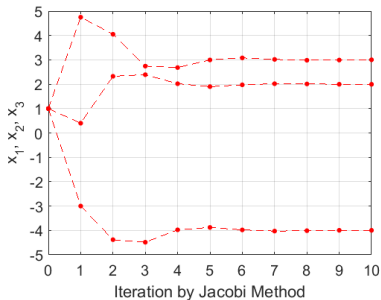
直接计算可以验证 $\rho(\mathcal{G}) = 0.2 < 1$, 故高斯-赛德尔迭代法收敛. 取初始值为 $x^{(0)} = [1, 1, 1]^\top$, 可得

迭代	0	1	...	4	5	...
分量 x_1	1	-3.0000	...	-3.9942	-4.0024	...
分量 x_2	1	3.7500	...	3.0058	2.9991	...
分量 x_3	1	2.0250	...	2.0006	2.0002	...

第4次迭代就已经得到原方程较好的近似解(准确解为 $[-4, 3, 2]^\top$).

计算结果比较

(显然, 这里的G-S迭代法收敛更快!)



超松弛迭代法

加权平均

超松弛迭代法

基本方法与主要结论

选取分裂矩阵 M 为带参数 ω 的下三角矩阵

$$M = \frac{1}{\omega}(D - \omega L) = \frac{1}{\omega}D - L,$$

其中 ω 为可选择的松弛因子.

构造迭代矩阵为(已讨论过一般情形: $B = I - M^{-1}A$)

$$L_{\omega} = I - \omega(D - \omega L)^{-1}A = (D - \omega L)^{-1}[(1 - \omega)D + \omega U],$$

从而得到解 $Ax = b$ 的逐次超松弛迭代法(SOR):

$$\begin{cases} x^{(0)}, & \text{初始向量, 一般取零向量,} \\ x^{(k+1)} = L_{\omega}x^{(k)} + f, & k = 0, 1, 2, \dots \end{cases}$$

其中 $L_{\omega} = (D - \omega L)^{-1}[(1 - \omega)D + \omega U]$, $f = \omega(D - \omega L)^{-1}b$.

超松弛迭代法

分量计算表达式

记 $x^{(k)} = \left(x_1^{(k)}, \dots, x_i^{(k)}, \dots, x_n^{(k)} \right)^\top$, 则

$$(D - \omega L)x^{(k+1)} = \left((1 - \omega)D + \omega U \right)x^{(k)} + \omega b,$$

或

$$Dx^{(k+1)} = Dx^{(k)} + \omega \left(b + Lx^{(k+1)} + Ux^{(k)} - Dx^{(k)} \right).$$

于是, SOR的计算表达式为

$$\begin{cases} x^{(0)} = \left(x_1^{(0)}, \dots, x_i^{(0)}, \dots, x_n^{(0)} \right)^\top, & \text{初始向量,} \\ x_i^{(k+1)} = x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i}^n a_{ij}x_j^{(k)} \right), & k = 0, 1, \dots \\ \omega \text{ 为松弛因子,} & i = 1, 2, \dots, n. \end{cases}$$

超松弛迭代法

SOR算法思想

- 当 $\omega = 1$ 时, SOR方法就是高斯-赛德尔迭代法;
- SOR方法的主要运算量来自矩阵与向量相乘(大部分迭代法都类似);
- 当 $\omega > 1$ 时, SOR称为超松弛法; 当 $\omega < 1$ 时, SOR称为低松弛法;
- 在编写计算机程序时, 可用

$$\max_{1 \leq i \leq n} |\Delta x_i| = \max_{1 \leq i \leq n} |x_i^{(k+1)} - x_i^{(k)}| < \varepsilon$$

标识迭代终止. 也可以用残差

$$\|r^{(k)}\|_{\infty} = \|b - Ax^{(k)}\|_{\infty} < \varepsilon$$

来控制迭代终止.

超松弛迭代法

SOR算法思想

实际上, SOR算法可以认为是高斯-赛德尔迭代法的一种直接修正:

- 假设 $x^{(k)}$ 和 $x_j^{(k+1)}$ ($j = 1, 2, \dots, i-1$)已经获得,
- 利用高斯-赛德尔迭代法计算中间辅助变量

$$\tilde{x}_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i}^n a_{ij} x_j^{(k)} \right),$$

- 利用 $x_i^{(k)}$ 与 $\tilde{x}_i^{(k+1)}$ 加权平均定义 $x_i^{(k+1)}$:

$$x_i^{(k+1)} = (1 - \omega) x_i^{(k)} + \omega \tilde{x}_i^{(k+1)} = x_i^{(k)} + \omega \left(\tilde{x}_i^{(k+1)} - x_i^{(k)} \right).$$

- 组合以上步骤, 便得到SOR算法.

超松弛迭代法

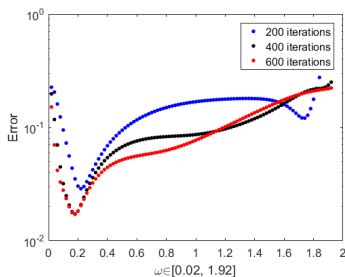
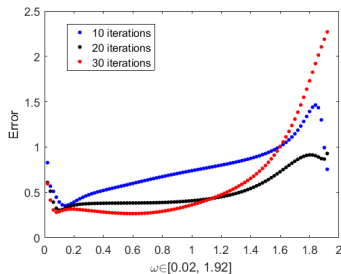
取 $n = 6$, 分别迭代10,20,30次.

求解

$$H_n x = b,$$

其中 H_n 是Hilbert矩阵,

$$H_n = (h_{ij}) \in \mathbb{R}^{n \times n}, \quad h_{ij} = \frac{1}{i+j-1}, \quad i, j = 1, 2, \dots, n.$$

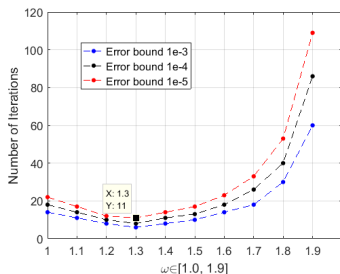
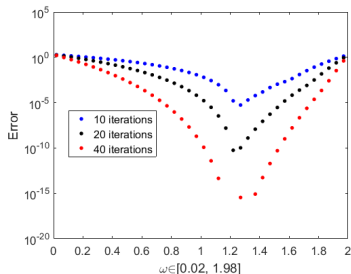


超松弛迭代法

SOR分别迭代10,20,40次.

求解

$$\begin{bmatrix} -4 & 1 & 1 & 1 \\ 1 & -4 & 1 & 1 \\ 1 & 1 & -4 & 1 \\ 1 & 1 & 1 & -4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}.$$



超松弛迭代法的收敛性

某些特殊类型矩阵对应的结果

定理1 考虑线性方程组 $Ax = b$, 如果

- (1) A 为对称正定矩阵, $A = D - L - U$;
- (2) $0 < \omega < 2$.

则求解 $Ax = b$ 的SOR迭代法收敛.

定理2 考虑线性方程组 $Ax = b$, 如果

- (1) A 为严格对角占优矩阵(或, A 为弱对角占优不可约矩阵);
- (2) $0 < \omega \leq 1$.

则求解 $Ax = b$ 的SOR迭代法收敛.

共轭梯度法

共轭梯度法：本质是直接法

共轭梯度法(CG: Conjugate Gradient method)

理论基础概述(核心思想是将“求解线性方程组”转化为“求解变分问题”)

设 $A = (a_{ij}) \in \mathbb{R}^{n \times n}$ 是**对称正定矩阵**, $b = [b_1, b_2, \dots, b_n]^\top$.
考虑 $Ax = b$. 引入如下二次函数 $\varphi: \mathbb{R}^n \rightarrow \mathbb{R}$:

$$\varphi(x) = \frac{1}{2}(Ax, x) - (b, x) = \frac{1}{2} \sum_{i,j=1}^n a_{ij} x_i x_j - \sum_{j=1}^n b_j x_j.$$

不难验证, 函数 φ 满足如下性质:

(1) 对一切 $x \in \mathbb{R}^n$, 函数 $\varphi(x)$ 的梯度为

$$\nabla \varphi(x) = Ax - b.$$

(2) 对一切 $x, y \in \mathbb{R}^n$ 和 $\alpha \in \mathbb{R}$,

$$\begin{aligned} \varphi(x + \alpha y) &= \frac{1}{2} \left(A(x + \alpha y), x + \alpha y \right) - (b, x + \alpha y) \\ &= \varphi(x) + \alpha(Ax - b, y) + \frac{\alpha^2}{2}(Ay, y). \end{aligned}$$

共轭梯度法

(3) 设 $x^* = A^{-1}b$ 是线性方程组 $Ax = b$ 的解, 则

$$\varphi(x^*) = -\frac{1}{2}(b, A^{-1}b) = -\frac{1}{2}(Ax^*, x^*),$$

且对一切 $x \in \mathbb{R}^n$, 有

$$\begin{aligned}\varphi(x) - \varphi(x^*) &= \frac{1}{2}(Ax, x) - \overbrace{(Ax^*, x)}^{Ax^*=b} + \frac{1}{2}(Ax^*, x^*) \\ &= \frac{1}{2}(A(x - x^*), x - x^*).\end{aligned}$$

定理3 设 A 对称正定. 则 x^* 是线性方程组 $Ax = b$ 解的充分必要条件是 x^* 满足

$$\varphi(x^*) = \min_{x \in \mathbb{R}^n} \varphi(x).$$

共轭梯度法

从定理3可知: 求解 $Ax = b$ 就是要构造一个向量序列 $\{x^{(k)}\}$, 使得 $\varphi(x^{(k)}) \rightarrow \varphi(x^*)$.

定理3的简单证明: 将方程组 $Ax = b$ 的解表示为 $x = A^{-1}b$. 由性质(3)与矩阵 A 的正定性可知

$$\varphi(x) - \varphi(x^*) = \frac{1}{2} \left(A(x - x^*), x - x^* \right) \geq 0.$$

所以对一切 $x \in \mathbb{R}^n$, 都成立 $\varphi(x) \geq \varphi(x^*)$. 这就是说 x^* 使 $\varphi(x)$ 取到最小.

反过来, 若有 x_1 也使 $\varphi(x)$ 取到最小, 则 $\varphi(x) \geq \varphi(x_1)$ 也对任意 $x \in \mathbb{R}^n$ 成立. 于是

$$\varphi(x_1) - \varphi(x^*) = \frac{1}{2} \left(A(x_1 - x^*), x_1 - x^* \right) = 0.$$

因为 A 是正定矩阵, 这导致 $x_1 = x^*$.

最速下降法

最速下降方向:

每一步选择负梯度方向

最速下降法

共轭方向相互正交

将求 $\varphi(x)$ 的极小点 x^* 转化为求一维问题的极小: 从某一个 $x^{(0)}$ 出发, 找一个方向 $p^{(0)}$, 作 $x^{(1)} = x^{(0)} + \alpha p^{(0)}$, 使得

$$\varphi(x^{(1)}) = \min_{\alpha \in \mathbb{R}} \varphi(x^{(0)} + \alpha p^{(0)}).$$

一般化, 便可以不断进行如下步骤:

(1) 记 $x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$, $k = 0, 1, 2, \dots$;

(2) 求 $\varphi(x^{(k+1)}) = \min_{\alpha \in \mathbb{R}} \left\{ \varphi(x^{(k)} + \alpha p^{(k)}) \right\}$;

称 $p^{(k)}$ 为下降方向(或搜索方向), α_k 为步长. 考虑第 $k+1$ 次搜索迭代, 二次函数 $\varphi(x^{(k+1)})$ 的表达式为

$$\begin{aligned} \varphi(x^{(k)} + \alpha p^{(k)}) &= \frac{1}{2} \left(A(x^{(k)} + \alpha p^{(k)}), (x^{(k)} + \alpha p^{(k)}) \right) \\ &\quad - \left(b, (x^{(k)} + \alpha p^{(k)}) \right) \end{aligned}$$

最速下降法

共轭方向相互正交

将 $\varphi(x^{(k)} + \alpha p^{(k)})$ 视为 α 的函数, 则 φ 取到极值, 意味着

$$\frac{d\varphi(x^{(k)} + \alpha p^{(k)})}{d\alpha} = 0.$$

于是(这是关于 α 的二次函数)

$$\varphi(x^{(k)} + \alpha p^{(k)}) = \varphi(x^{(k)}) + \alpha(Ax^{(k)} - b, p^{(k)}) + \frac{\alpha^2}{2}(Ap^{(k)}, p^{(k)}),$$

求导可知

$$\frac{d}{d\alpha}\varphi(x^{(k)} + \alpha p^{(k)}) = (Ax^{(k)} - b, p^{(k)}) + \alpha(Ap^{(k)}, p^{(k)}) = 0,$$

最速下降法

共轭方向相互正交

这样就得到第 k 步计算的最优“步长”为

$$\alpha_k = -\frac{(Ax^{(k)} - b, p^{(k)})}{(Ap^{(k)}, p^{(k)})}.$$

由于 α_k 使得 φ 取到极小, 故而有

$$\varphi(x^{(k)} + \alpha_k p^{(k)}) \leq \varphi(x^{(k)} + \alpha p^{(k)}), \quad \forall \alpha \in \mathbb{R}.$$

- 选择一个恰当的方向 $p^{(k)}$, 使得 $\varphi(x)$ 在点 $x^{(k)}$ 沿着方向 $p^{(k)}$ 下降最快;
- 上述方向实际上是负梯度方向:

$$\nabla \varphi(x^{(k)}) = - \left(\frac{\partial \varphi(x^{(k)})}{\partial x_1}, \frac{\partial \varphi(x^{(k)})}{\partial x_2}, \dots, \frac{\partial \varphi(x^{(k)})}{\partial x_n} \right)^{\top}.$$

- $p^{(k)} = -\nabla \varphi(x^{(k)}) = -(Ax^{(k)} - b) = r^{(k)}.$

最速下降法

从现在开始, 为了书写方便, 将下降方向 p, r 的上标改写为下标.

现在, 最优“步长”可以表示为(将 $r^{(k)}$ 记为 r_k)

$$\alpha_k = \frac{(r^{(k)}, r^{(k)})}{(Ar^{(k)}, r^{(k)})} = \frac{r_k^\top r_k}{r_k^\top Ar_k}.$$

于是

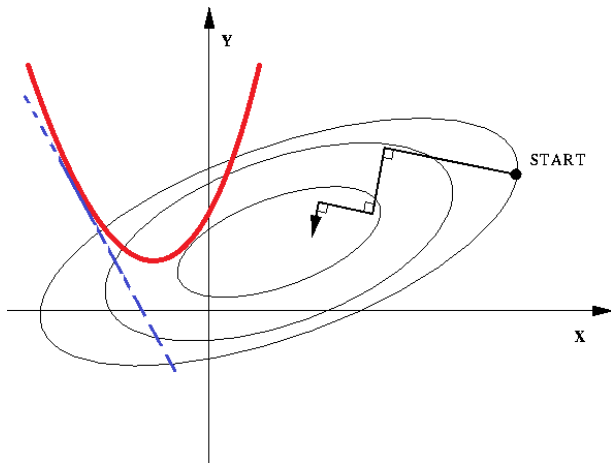
$$x^{(k+1)} = x^{(k)} + \alpha_k r^{(k)}, \quad k = 0, 1, 2, \dots,$$

其中 $r^{(k)} = b - Ax^{(k)}$ 为第 k 步计算的剩余向量. 进一步计算还可以看到

$$\begin{aligned} (r^{(k+1)}, r^{(k)}) &= (b - A(x^{(k)} + \alpha_k r^{(k)}), r^{(k)}) \\ &= (r^{(k)}, r^{(k)}) - \alpha_k (Ar^{(k)}, r^{(k)}) \\ &= 0, \end{aligned}$$

这说明连续两次计算的搜索方向是正交的.

最速下降法



最速下降法

共轭方向相互正交

此时,

$$\begin{aligned}\varphi(x^{(k+1)}) - \varphi(x^{(k)}) &= \varphi(x^{(k)} + \alpha_k r_k) - \varphi(x^{(k)}) \\ &= -\alpha_k r_k^\top r_k + \frac{1}{2} \alpha_k^2 r_k^\top A r_k \\ &= -\frac{1}{2} \frac{[r_k^\top r_k]^2}{r_k^\top A r_k} < 0,\end{aligned}$$

这说明当 $k \rightarrow \infty$ 时, $x_k \rightarrow x^*$, $x^* = A^{-1}b$ 是线性方程组的解.
不加证明, 陈述如下结论:

$$\|x^{(k)} - x^*\|_A \leq \left(\frac{\lambda_1 - \lambda_n}{\lambda_1 + \lambda_n} \right)^k \|x^{(0)} - x^*\|_A,$$

其中 λ_1 和 λ_n 分别为对称正定矩阵 A 的最大与最小特征值. 显然
当 $\lambda_1 \gg \lambda_n$ 时, 最速下降法收敛是很慢的.

最速下降法：简单算例

例1 求解 $3x = 1$, 不难知道 $x = \frac{1}{3}$.

事实上, 为使用最速下降法, 构造二次函数

$$\varphi(x) = \frac{3}{2}x^2 - x, \quad x \in \mathbb{R},$$

接下来就是从初始点 $x_0 = 0$ 开始迭代, 计算函数 $\varphi(x)$ 的极小值点. 根据一元函数极值的必要条件, $\varphi(x)$ 取极小值时满足 $\varphi'(x) = 0$, 即 $3x - 1 = 0$.

在 $x = 0$ 时的下降方向满足 $r_0 = 1 - 3x|_{x_0} = 1 = p_0$, 于是

$$x_1 = x_0 + \alpha p_0 = \alpha,$$

代入二次函数可知

$$\varphi(x_1) = \varphi(\alpha) = \frac{3}{2}\alpha^2 - \alpha.$$

最速下降法: 简单算例

若要 $\varphi(x_1)$ 取极小, 由微积分知识可得

$$\alpha = \frac{1}{3}.$$

于是 $x_1 = \frac{1}{3}$. 因二次函数 $\varphi(x)$ 是一元函数, 故在一个方向上搜索得到最优即就是全局最优.

例2 求解 $Ax = b$, 其中 $A = \begin{bmatrix} 2 & 0 \\ 0 & 2 \end{bmatrix}$, $b = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$.

为使用最速下降法, 记二元二次函数 $\varphi(x)$ 为

$$\varphi(x) = \varphi(x_1, x_2) = \frac{1}{2}(Ax, x) - (b, x) = x_1^2 + x_2^2 - (x_1 + x_2).$$

由Lagrange乘子法立刻知最优解(即原方程的解)为 $x_1 = x_2 = \frac{1}{2}$.

最速下降法: 简单算例

取 $x_0 = [0, 0]^\top$, 下降方向为 p_0 , 步长为 α_0 . 则

$$\nabla\varphi(x)|_{x=x_0} = \begin{bmatrix} 2x_1 - 1 \\ 2x_2 - 1 \end{bmatrix} = \begin{bmatrix} -1 \\ -1 \end{bmatrix} = -p_0.$$

设 $x_1 = x_0 + \alpha_0 p_0$, 于是

$$\varphi(x_1) = (\alpha_0^1)^2 + (\alpha_0^2)^2 - (\alpha_0^1 + \alpha_0^2) := \varphi(\alpha_0^1, \alpha_0^2).$$

考虑二元函数 $\varphi(\alpha_0^1, \alpha_0^2)$ 不难发现: 当

$$\alpha_0^1 = \alpha_0^2 = \frac{1}{2}$$

时, $\varphi(\alpha)$ 取得极值. 于是 $x_1 = [1/2, 1/2]^\top$. 经检验, 在 x_1 处

$$\nabla\varphi(x_1) = 0,$$

这意味着 x_1 就是全局最优解, 也是所求线性方程组的解.

最速下降法上机实践

首先确定线性方程组解的存在唯一性

考虑如下线性方程组

$$\begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

- 用最速下降法迭代;
- 系数矩阵特征值为 $\lambda_1 = 4$, $\lambda_2 = 1$;
- 迭代方向可以直观绘制;
- 真解为 $x^* = \begin{bmatrix} 2 \\ -\frac{3}{2} \end{bmatrix}$;
- 可深入比较系数矩阵对称正定与一般情形时, 最速下降法的收敛行为.

最速下降法上机实践

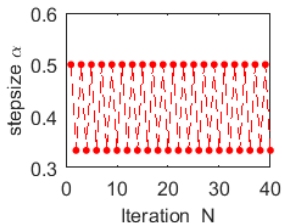
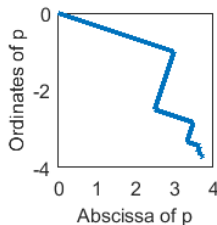
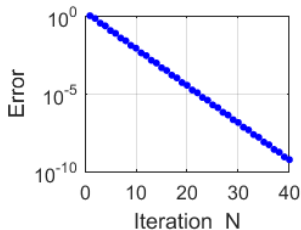
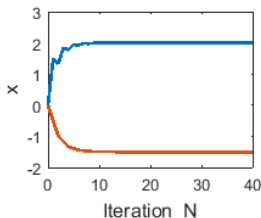
比较迭代结果, 误差, 每步迭代方向和步长变化情况

- 真解为 $x^* = [2, -1.5]^\top$, 初值取 $[0, 0]^\top$;
- 最速下降法迭代15次结果如下

k	x_1	x_2	p_1	p_2	α
0	0	0	3.0000	-1.0000	0.5000
1	1.5000	-0.5000	-0.5000	-1.5000	0.3333
2	1.3333	-1.0000	1.0000	-0.3333	0.5000
3	1.8333	-1.1667	-0.1667	-0.5000	0.3333
4	1.7778	-1.3333	0.3333	-0.1111	0.5000
5	1.9444	-1.3889	-0.0556	-0.1667	0.3333
6	1.9259	-1.4444	0.1111	-0.0370	0.5000
7	1.9815	-1.4630	-0.0185	-0.0556	0.3333
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$
14	1.9991	-1.4993	-0.0007	-0.0021	0.5000
15	1.9998	-1.4995	0.0014	-0.0005	

最速下降法上机实践

数值结果, 误差, 每步迭代方向, 步长变化情况



最速下降法上机实践

比较迭代结果, 误差, 每步迭代方向和步长变化情况

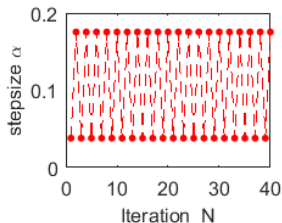
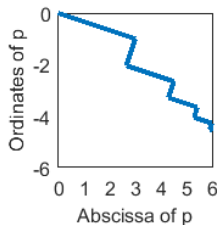
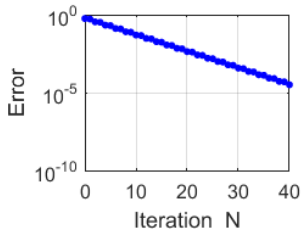
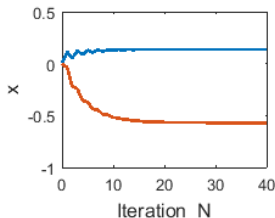
考虑如下线性方程组

$$\begin{bmatrix} 30 & 2 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

- 真解为 $x^* = \begin{bmatrix} 0.1379 \\ -0.5690 \end{bmatrix}$;
- 系数矩阵特征值为 $\lambda_1 = 30.0712$, $\lambda_2 = 1.9288$;
- 特征值相差较大, 收敛速度明显较慢;
- 用最速下降法迭代40次.

最速下降法上机实践

比较迭代结果, 误差, 每步迭代方向和步长变化情况



共轭梯度法(born in 1952)

[https://cn.bing.com/images/search?q=conjugate gradient method](https://cn.bing.com/images/search?q=conjugate+gradient+method)

共轭梯度法

共轭梯度法(born in 1952)

[https://cn.bing.com/images/search?q=conjugate gradient method](https://cn.bing.com/images/search?q=conjugate+gradient+method)

Conjugate Gradient Methods



Magnus Hestenes (1906-1991)



Eduard Stiefel (1909-1978)

$$x_{k+1} = x_k + \alpha_k d_k$$

$$d_{k+1} = -g_{k+1} + \beta_k s_k$$

$$s_k = x_{k+1} - x_k$$

$$y_k = g_{k+1} - g_k$$

共轭梯度法：共轭方向

前面已经说明, 连续两次选择的下降方向 $\{r_k, r_{k-1}\}_{k=1}^{\infty}$ 正交. 于是, 可以尝试构造一组“**A-共轭**”(或称A-正交)的向量族

$$\mathbb{R}^n \supset \text{span} \{p_0, p_1, \cdots, p_n\}.$$

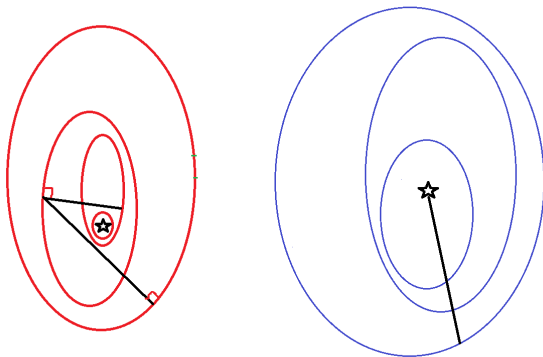
即向量族具有性质 $p_i^\top A p_j = 0$ ($i \neq j$). 尽管方向 $\{p^{(k)}\}$ 不如 r_k 正交, 但具有更好的其他性质:

- 假设已经计算得到 $x^{(k+1)} = x^{(k)} + \alpha_k p^{(k)}$,
其中 $x^k = \alpha_0 p_0 + \alpha_1 p_1 + \cdots, \alpha_{k-1} p_{k-1}$,
- 初始计算取 $p_0 = r_0 = b - Ax^{(0)}$,
- 之后每一次确定 p_k , 都使得 $p_k^\top A p_j = 0$ ($k \neq j$).

优点

- 理论来说, 只需有限步计算获得 $Ax = b$ 的准确解;
- 本质上, 共轭梯度法是在有限维空间中直接求解方程组.

最速下降方向 v.s. 共轭方向 等高线图



共轭梯度法

理论分析

初始化: 取 $p_0 = r_0$. 当 $k \geq 1$ 时, 确定 p_k 除了满足

$$\varphi(x^{(k+1)}) = \min_{\alpha \in \mathbb{R}} \varphi(x^{(k)} + \alpha p_k),$$

还希望 p_k 满足

$$\varphi(x^{(k+1)}) = \min_{x \in \text{span}\{p_0, p_1, \dots, p_k\}} \varphi(x),$$

其中 $x \in \text{span}\{p_0, p_1, \dots, p_k\}$ 表示为

$$x = y + \alpha p_k, \quad y \in \text{span}\{p_0, p_1, \dots, p_{k-1}\}, \quad \alpha \in \mathbb{R}.$$

共轭梯度法

理论分析

所以

$$\begin{aligned}\varphi(x) &= \varphi(y + \alpha p_k) \\ &\quad \text{该交叉项必须为零} \\ &= \varphi(y) + \overbrace{\alpha(Ay, p_k)} - \alpha(b, p_k) + \frac{\alpha^2}{2}(Ap_k, p_k)\end{aligned}$$

进一步将极小化 φ 的问题转化为两个极小化问题:

$$\begin{aligned}\min_{x \in \text{span}\{p_0, p_1, \dots, p_k\}} \varphi(x) &= \min_{\alpha, y} \varphi(x^{(k)} + \alpha p_k) \\ &= \min_y \varphi(y) \\ &\quad + \min_{\alpha} \left[\frac{\alpha^2}{2}(Ap_k, p_k) + \alpha(Ay, p_k) - \alpha(b, p_k) \right].\end{aligned}$$

- 蓝色部分: 通过极小 $y \in \text{span}\{p_0, p_1, \dots, p_{k-1}\}$ 解出 $y = x^{(k)}$.

共轭梯度法

理论分析

- 红色部分: 由 $r_k = b - Ax^{(k)}$ 以及 α_k 的表达式, 可得

$$\alpha_k = \frac{(r_k, p_k)}{(Ap_k, p_k)}. \quad (19)$$

考虑到 p_k 选为 $\{p_0, p_1, \dots, p_{k-1}\}$ 的 A -共轭时并不唯一, 故采用 r_k 与 p_{k-1} 的某种线性组合. 比如:

$$p_k = r_k + \beta_{k-1} p_{k-1},$$

利用 A -共轭定义 $(p_k, Ap_{k-1}) = 0$ 可求出

$$\beta_{k-1} = -\frac{(r_k, Ap_{k-1})}{(p_{k-1}, Ap_{k-1})}, \quad (20)$$

这样一来, 确定的 p_k 和 p_{k-1} 就是 A -共轭的.

共轭梯度法

理论分析: 简化系数 α_k 与 β_k 的计算过程.

计算 α_k 的表达式(19)可以简化: 由于

$$r_{k+1} = b - Ax^{(k+1)} = b - A[x^{(k)} + \alpha_k p_k] = r_k - \alpha_k A p_k,$$

同时, 第 k 步的共轭方向与第 $k+1$ 步的最速下降方向满足

函数 $\varphi(x^{(k)} + \alpha_k p_k)$ 对 α_k 的导函数

$$(r_{k+1}, p_k) = \overbrace{(r_k, p_k) - \alpha_k (A p_k, p_k)}^{\text{函数}\varphi(x^{(k)} + \alpha_k p_k)\text{对}\alpha_k\text{的导函数}} = 0$$

于是,

$$(r_k, p_k) = (r_k, r_k + \beta_{k-1} p_{k-1}) = (r_k, r_k) + \beta_{k-1} (r_k, p_{k-1}) = (r_k, r_k).$$

代入(19)即可化简其为

$$\alpha_k = \frac{(r_k, r_k)}{(A p_k, p_k)}. \quad (21)$$

不难看到, 当 r_k 非零(迭代计算未完成)时, $\alpha_k > 0$, 迭代计算仍将继续.

共轭梯度法

理论分析: 简化系数 α_k 与 β_k 的计算过程.

计算 β_k 的表达式(20)也可以简化. 注意到 r_k 是最速下降方向, 故而 $(r_k, r_j) = 0$ ($k \neq j$); 同时共轭方向满足A-正交性:
 $(Ap_k, p_j) = (p_k, Ap_j) = 0$ ($k \neq j$). 于是

$$r_{k+1} = r_k - \alpha_k Ap_k \Rightarrow Ap_k = \alpha_k^{-1}(r_k - r_{k-1}),$$

代入(20)可得

$$\begin{aligned}\beta_k &= -\frac{(r_{k+1}, Ap_k)}{(p_k, Ap_k)} = -\frac{(r_{k+1}, \alpha_k^{-1}(r_k - r_{k-1}))}{(r_k + \beta_{k-1}p_{k-1}, Ap_k)} \\ &= \frac{(r_{k+1}, r_{k+1})}{\alpha_k(r_k, Ap_k)} = \frac{(r_{k+1}, r_{k+1})}{\alpha_k(r_k, \alpha_k^{-1}(r_k - r_{k-1}))} \\ &= \frac{(r_{k+1}, r_{k+1})}{(r_k, r_k)}.\end{aligned}\tag{22}$$

不难看到, 当 r_{k+1} 非零(迭代计算未完成)时, $\beta_k > 0$, 迭代计算仍将继续.

共轭梯度法: CG算法流程

- (1) 任取 $x^{(0)} \in \mathbb{R}^n$, 计算 $r_0 = b - Ax^{(0)}$, 取 $p_0 = r_0$;
- (2) 对 $k = 0, 1, 2, \dots$, 计算

$$\alpha_k = \frac{(r_k, r_k)}{(p_k, Ap_k)},$$
$$x^{(k+1)} = x^{(k)} + \alpha_k p_k,$$

$$r_{k+1} = r_k - \alpha_k Ap_k, \quad \beta_k = \frac{(r_{k+1}, r_{k+1})}{(r_k, r_k)},$$

$$p_{k+1} = r_{k+1} + \beta_k p_k.$$

- (3) 若 $r_k = 0$ 或 $(p_k, Ap_k) = 0$, 计算停止, 则 $x^{(k)} = x^*$; 由于 A 正定, 故当 $(p_k, Ap_k) = 0$ 时, $p_k = 0$, 而 $(r_k, r_k) = (r_k, p_k) = 0$, 即就是 $r_k = 0$ (残差为零, 得到方程组的解).

共轭梯度法: CG算法流程

```
for k = 1:N
    if k==1
        r(:,k) = b - A*x(:,k);
        p(:,k) = r(:,k);
    end
    alpha(k) = sum(r(:,k).^2)/((A*p(:,k))'*p(:,k));
    x(:,k+1) = x(:,k) + alpha(k)*p(:,k);
    r(:,k+1) = r(:,k) - alpha(k)*A*p(:,k);
    beta(k) = sum(r(:,k+1).^2)/(sum(r(:,k).^2));
    p(:,k+1) = r(:,k+1) + beta(k)*p(:,k);
end
```

共轭梯度法

数值算例

应用共轭梯度法解方程组

$$\begin{cases} 2x_1 + x_3 = 3, \\ x_2 = 1, \\ x_1 + 2x_3 = 3. \end{cases}$$

不难看到, 真解为 $[1, 1, 1]^\top$. 系数矩阵

$$A = \begin{bmatrix} 2 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 2 \end{bmatrix}$$

是实对称正定的. 取初始值为 $[0, 0, 0]^\top$, 根据CG方法的计算步骤, 可知

$$r_0 = b - Ax^{(0)} = [3, 1, 3]^\top, \quad p_0 = r_0.$$

共轭梯度法

数值算例

当 $k = 0$ 时, 简单计算可知

$$Ap_0 = [9, 1, 9]^\top, \quad p_0^\top Ap_0 = 55.$$

因此

$$\alpha_0 = -\frac{r_0^\top p_0}{p_0^\top Ap_0} = \frac{19}{55},$$

$$x^{(1)} = x^{(0)} + \alpha_0 p_0 = \frac{19}{55}[3, 1, 3]^\top,$$

$$r_1 = r_0 + \alpha_0 Ap_0 = \frac{6}{55}[1, -6, 1]^\top,$$

$$\beta_0 = \frac{r_1^\top Ap_0}{p_0^\top Ap_0} = \frac{6 \times 12}{55 \times 55},$$

$$p_1 = -r_1 + \beta_0 p_0 = \frac{6 \times 19}{55 \times 55}[-1, 18, -1]^\top.$$

共轭梯度法

数值算例

当 $k = 1$ 时, 继续计算可知

$$Ap_1 = \frac{6 \times 3 \times 19}{55 \times 55} [-1, 6, -1]^\top, \quad p_1^\top Ap_1 = \frac{6 \times 6 \times 3 \times 19 \times 19 \times 2}{55 \times 55 \times 55}.$$

因此

$$r_1^\top p_1 = -\frac{6 \times 6 \times 19 \times 2}{55 \times 55},$$

$$\alpha_1 = -\frac{r_1^\top p_1}{p_1^\top Ap_1} = \frac{55}{3 \times 19},$$

$$x^{(2)} = x^{(1)} + \alpha_1 p_1 = [1, 1, 1]^\top,$$

$$r_2 = Ax^{(2)} - b = 0.$$

这说明第二次迭代结果 $x^{(2)}$ 就为 $Ax = b$ 的准确解. 注意: 实际计算时由于可能的舍入误差将导致共轭梯度法并非总是在有限步给出精确值.

共轭梯度法计算实例

比较迭代结果, 误差, 每步迭代方向和步长变化情况

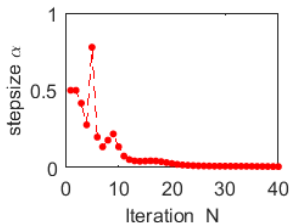
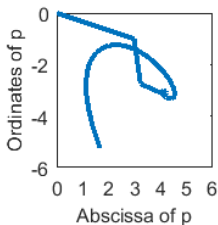
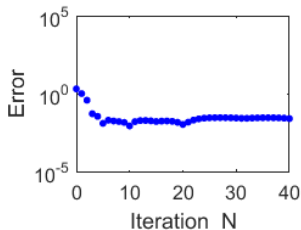
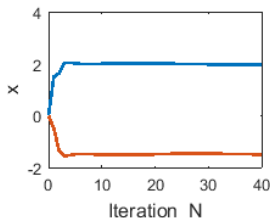
考虑如下线性方程组

$$\begin{bmatrix} 3 & 2 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

- 方程组的系数矩阵不是大型稀疏对称正定矩阵;
- 真解为 $x^* = [2, -1.5]^T$;
- 系数矩阵特征值为 $\lambda_1 = 4, \lambda_2 = 1$;
- 迭代方向可以直观绘制;
- 用共轭梯度法迭代40次.
- 将弱对角占优矩阵预处理为对称正定: 用 $\begin{bmatrix} 3 & 1 \\ 2 & 2 \end{bmatrix}$ 左乘方程的两端, 将系数矩阵变成对称正定, 然后采用共轭梯度法, 可在两步迭代给出准确结果.

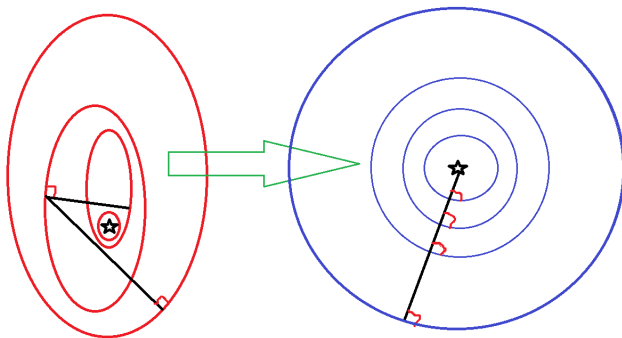
共轭梯度法计算实例

系数矩阵不是对称正定, 计算结果不理想. 思考: 原因有哪些?



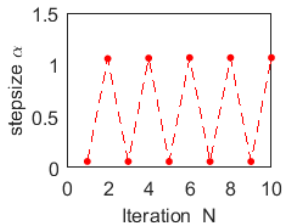
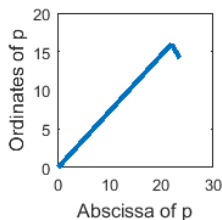
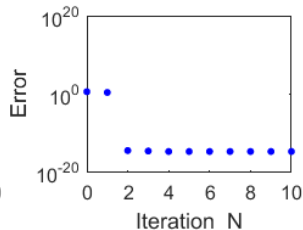
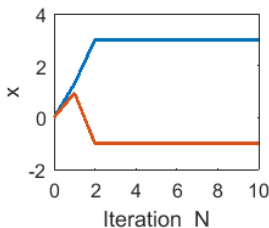
共轭梯度法计算实例

系数矩阵不是对称正定, 计算结果不理想. 思考: 原因有哪些?



共轭梯度法计算实例

预处理后, 系数矩阵对称正定, 迭代两步达到机器精度.



共轭梯度法计算实例

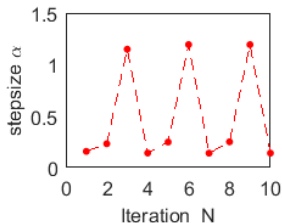
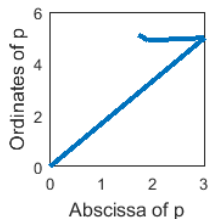
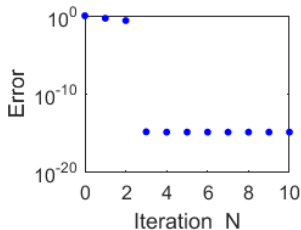
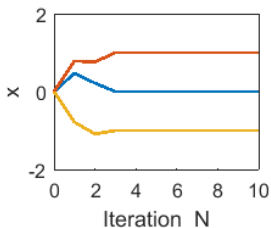
考虑如下线性方程组(210页, 10(2))

$$\begin{bmatrix} 4 & 3 & 0 \\ 3 & 4 & -1 \\ 0 & -1 & 4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} 3 \\ 5 \\ -5 \end{bmatrix}$$

- 方程组的系数矩阵是对称正定矩阵;
- 真解为 $x^* = [0, 1, -1]^\top$;
- 系数矩阵特征值为 $\lambda_1 = 7.1623$, $\lambda_2 = 4$, $\lambda_3 = 0.8377$;
- 用共轭梯度法迭代10次;
- 迭代3次获得准确解.

共轭梯度法计算实例

系数矩阵对称正定, 迭代三步达到机器精度.



共轭梯度法计算实例

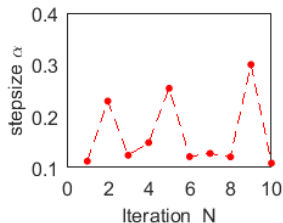
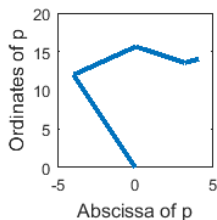
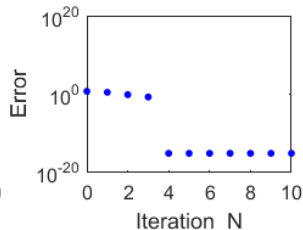
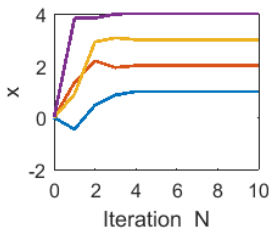
考虑如下线性方程组

$$\begin{bmatrix} 5 & -1 & -1 & -1 \\ -1 & 10 & -1 & -1 \\ -1 & -1 & 5 & -1 \\ -1 & -1 & -1 & 10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} = \begin{bmatrix} 4 \\ 12 \\ 8 \\ 34 \end{bmatrix}$$

- 方程组的系数矩阵是对称正定矩阵;
- 真解为 $x^* = [1, 2, 3, 4]^T$;
- 系数矩阵特征值为 $\lambda_1 = 11.0000$, $\lambda_2 = 9.7016$, $\lambda_3 = 6.0000$, $\lambda_4 = 3.2984$;
- 用共轭梯度法迭代10次;
- 迭代4次获得准确解.

共轭梯度法计算实例

系数矩阵对称正定, 迭代四步达到机器精度.



共轭梯度法计算实例

考虑如下线性方程组

$$\begin{bmatrix} 5 & -1 & -1 & -1 & 1 & -1 & 1 & -1 \\ -1 & 10 & -1 & -1 & 1 & -1 & 1 & -1 \\ -1 & -1 & 5 & -1 & -1 & 1 & -1 & 1 \\ -1 & -1 & -1 & 10 & -1 & 1 & -1 & 1 \\ 1 & 1 & -1 & -1 & 5 & -1 & -1 & -1 \\ -1 & -1 & 1 & 1 & -1 & 10 & -1 & -1 \\ 1 & 1 & -1 & -1 & -1 & -1 & 5 & -1 \\ -1 & -1 & 1 & 1 & -1 & -1 & -1 & 10 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \\ x_8 \end{bmatrix} = \begin{bmatrix} -1 \\ 10 \\ 10 \\ 36 \\ -8 \\ 16 \\ 4 \\ 38 \end{bmatrix}$$

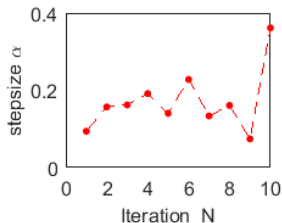
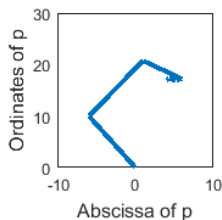
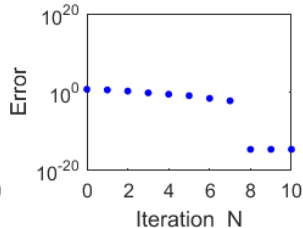
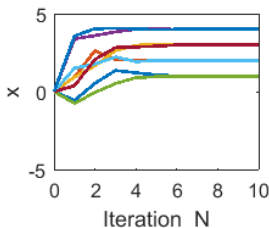
- 方程组的系数矩阵是对称正定矩阵;
- 真解为 $x^* = [1, 2, 3, 4, 1, 2, 3, 4]^T$;
- 系数矩阵特征值为 λ_k ($k = 1, 2, \dots, 8$)

$$\lambda_k = \{2.3909, 3.2984, 5.2321, 6, 8.8925, 9.7016, 11, 13.4845\};$$

- 用共轭梯度法迭代10次;

共轭梯度法计算实例

系数矩阵对称正定, 迭代八步达到机器精度.





预估-校正格式

中点公式+梯形公式

考虑

$$\frac{dy}{dx} = f(x, y), \quad 0 \leq x \leq L, \quad y(0) = y_0. \quad (23)$$

预估步骤: (中点公式)

$$y_{n+1} = y_{n-1} + 2hf_n, \quad (24)$$

校正步骤: (梯形公式)

$$y_{n+1} = y_n + \frac{h}{2} (f_n + f_{n+1}), \quad (25)$$

其中 $y_n = y(x_n)$, $f_n = f(x_n, y(x_n))$.

数值实验

算例2: (常微分方程)

考虑如下的Lotka-Volterra模型:

$$\begin{cases} \frac{dx}{dt} = ax - bxy + mx^2 - sx^2z = f_1(x, y, z), \\ \frac{dy}{dt} = -cy + dxy = f_2(x, y, z), \\ \frac{dz}{dt} = -pz + sx^2z = f_3(x, y, z), \end{cases} \quad (26)$$

其中系数 a, b, c, d 均为正. a 表示没有猎物的捕食者的自然增长率, b 表示捕食对猎物的影响, c 表示没有猎物的捕食者的自然死亡率, d 表示捕食者在有猎物情形下的影响和繁殖成活率, m, p, s 为正的参数值.

数值实验

为计算方便, 可取

$$a = 1, b = 1, c = 1, d = 1, m = 2, s = 2.7, p = 3,$$

初始条件为

$$[x_0, y_0, z_0] = [1.5, 1.5, 1.5].$$

记 x_n , y_n 和 z_n 分别表示函数 $x(t)$, $y(t)$ 和 $z(t)$ 在时刻 $t = t_n$ 的函数值.

数值格式

● 构造迭代格式

$$\begin{cases} x_{n+1} = x_n + h \times (ax_n - bx_n y_n + mx_n^2 - sx_n^2 z_n), \\ y_{n+1} = y_n + h \times (-cy_n + dx_n y_n), \\ z_{n+1} = y_n + h \times (-pz_n + sx_n^2 z_n), \end{cases} \quad (27)$$

其中 h 为步长. 一般地, 根据逼近导数的方式不同, 可以构造出很多不同的迭代格式. 比如**Jacobi迭代**, **Gauss-Seidal迭代**, **SOR迭代**, **共轭梯度迭代**, **Euler迭代**, **预条件迭代**, **预估校正迭代**...

数值格式

- 利用梯形公式进行校正

$$\begin{cases} x_{n+1} = x_n + \frac{h}{2} \times (f_{1,n} + f_{1,n+1}), \\ y_{n+1} = y_n + \frac{h}{2} \times (f_{2,n} + f_{2,n+1}), \\ z_{n+1} = y_n + \frac{h}{2} \times (f_{3,n} + f_{3,n+1}), \end{cases} \quad (28)$$

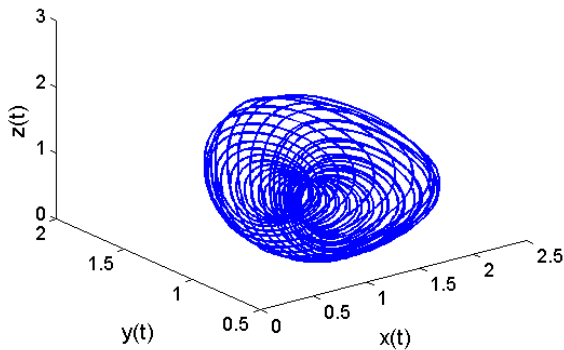
其中 h 为步长, $f_{i,n} = f_i(x_n, y_n, z_n)$, $i = 1, 2, 3$.

- 还可以用其他的求积公式进行校正, 详见“数值分析”或者“科学计算”等参考资料.

数值结果

取 $a = 1, b = 1, c = 1, d = 1, m = 2, s = 2.7, p = 3$:

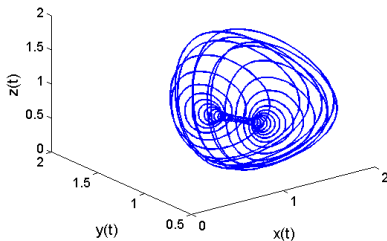
$p = 3$



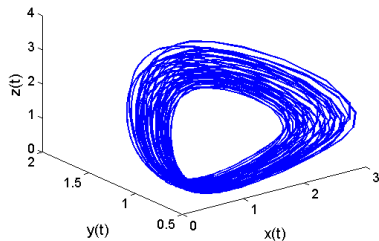
数值结果

其余不变, 改变 p 的值. 分别取 $p = 2.8$ 和 $p = 4$:

$p = 2.8$

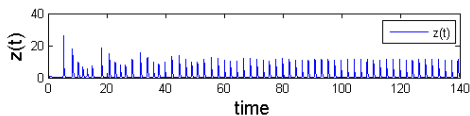
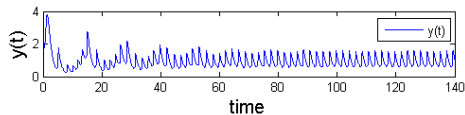
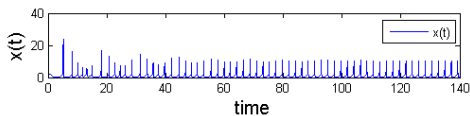
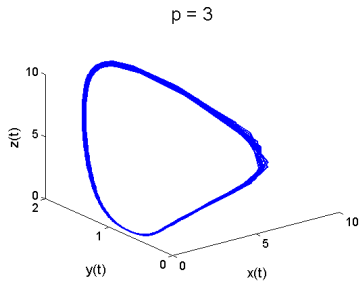


$p = 4$



数值结果

其余不变, 分别取 $s = p = 3$, 可得周期解如下:



问题与讨论

Thank You!